

浅谈计数类题目的常见技巧

||

重庆市巴蜀中学校

2026 年 4 月 13 日

ARC165E Random Isolation

给定一棵 n 个节点的树，节点初始为白色。不断的随机从中找出没有被涂黑的节点，如果其所在的白色连通块点数大于 k 则涂黑该点，无法操作时结束。求出期望涂黑的节点数。

数据范围： $1 \leq k \leq n \leq 100$ 。

- 提示 1: 尝试求出每一个节点被涂黑的概率, 根据期望的线性性, 相加即为答案。

ARC165E Random Isolation

- 提示 1: 尝试求出每一个节点被涂黑的概率, 根据期望的线性性, 相加即为答案。
- 提示 2: 尝试枚举一个节点所在的连通块的每一种情况, 如何计算概率?

ARC165E Random Isolation

- 提示 1: 尝试求出每一个节点被涂黑的概率, 根据期望的线性性, 相加即为答案。
- 提示 2: 尝试枚举一个节点所在的连通块的每一种情况, 如何计算概率?
- 提示 3: 使用 dp 优化。

ARC165E Random Isolation

我们考虑找到每一个节点被涂黑的概率，假设我们现在枚举了节点 u 以及其所在的连通块 S ，首先有 $|S| > k$ 。

再考虑与该连通块直接连接的，不在该连通块内的点，我们记作 T 。

可以发现，要使得该连通块合法，必定需要保证在 S 中任意节点出现前， T 中的数均出现至少一次。这里我们保证 T 中的节点一定涂黑的原因是与 T 相连的 S 大小大于 k 。

ARC165E Random Isolation

之后我们将会涂黑节点 u , u 之后的操作已经不会影响概率。
我们不妨将每一个数的首次出现时间从随机序列中提取出来, $n!$
种情况是均匀随机的。那么我们就可以通过组合计数求出概率:

$$\frac{|T|!(|S| - 1)!}{(|T| + |S|)!}.$$

我们发现该式子只和 $|T|$ 与 $|S|$ 有关, 于是我们可以使用树形 dp
进行计算, 设 $f_{u,i,j}$ 为在以 u 为根的子树内, u 包含在连通块内,
 $|S| = i$, $|T| = j$ 时的方案数。统计答案时在所有连通块的根处统
计块内所有点的贡献。

时间复杂度 $O(n^4)$ 。

QOJ9798 Safety First

T 组测试, 每次给定 n, m , 找到长度为 n 单调不升的正整数序列 d , 再找到 $\{d_2, \dots, d_n\}$ 的子集, 使得该子集和加上 d_1 等于 m , 求选择序列和子集的方案数。

数据范围: $1 \leq T \leq 10^5, 1 \leq n, m \leq 2000$ 。

QOJ9798 Safety First

- 提示 1: 尝试想到一个 n^3 做法。

QOJ9798 Safety First

- 提示 1: 尝试想到一个 n^3 做法。
- 提示 2: 尝试想到两个本质不同的 n^3 做法。

QOJ9798 Safety First

- 提示 1: 尝试想到一个 n^3 做法。
- 提示 2: 尝试想到两个本质不同的 n^3 做法。
- 提示 3: 尝试合并这两个 n^3 做法, 使得复杂度正确。

考虑一种简单的 n^3 dp: 设 $f_{i,j,k}$ 为枚举了前 i 个值, $d_i \geq j$, 选出的子集和加 d_1 为 k 的方案数。

转移可以考虑, 新增一个 $d = j$, 或者将 $j \leftarrow j - 1$:

$$f_{i,j,k} \leftarrow f_{i,j+1,k} + f_{i-1,j,k} + f_{i-1,j,k-j}.$$

初始状态为枚举 d_1 , $f_{1,d_1,d_1} \leftarrow 1$, 复杂度为 $O(n^3)$ 。

QOJ9798 Safety First

考虑一种不简单的 n^3 dp: 设 $g_{i,j,k}$ 为枚举了前 i 个值, 有 j 个值被选入了子集与 d_1 , 当前和为 k 的方案数。
 转移可以考虑, 新增一个 $d = 1$, 或者将前 i 个值集体加一, 可以发现此操作不会改变序列的单调性且计数不重不漏。

$$g_{i,j,k} \leftarrow g_{i-1,j,k} + g_{i-1,j-1,k+1} + g_{i,j,k-j}$$

初始状态为 $g_{1,1,1} \leftarrow 1$, 复杂度为 $O(n^3)$ 。

QOJ9798 Safety First

然后我们敏锐的发现，在选出子集总和等于 m 时，子集中 $\geq \sqrt{m}$ 的数的个数一定 $\leq \sqrt{m}$ ，这就代表了第二个 dp 中第二个维度的上界为 $\sqrt{m}$ 。

因此我们可以将 g 变动一下：新增值变为 $d = \sqrt{m}$ ，这样该 dp 的复杂度就变为了 $O(n^{2.5})$ 。

对于值 $< \sqrt{m}$ 的部分，可以使用第一个 dp， f 的初始值使用 g 即可。该部分复杂度也为 $O(n^{2.5})$ 。

ARC148E $\geq K$

给定长度为 n 的数列 a 和一个自然数 K , 将 a 重排为 a' , 问多少种 a' 满足 $\forall i \in [1, n), a'_i + a'_{i+1} \geq K$ 。

数据范围: $2 \leq n \leq 2 \times 10^5, 0 \leq a_i, K \leq 10^9$ 。

ARC148E $\geq K$

- 提示 1: 考虑讨论 a 与 $\frac{K}{2}$ 的大小关系。

ARC148E $\geq K$

- 提示 1: 考虑讨论 a 与 $\frac{K}{2}$ 的大小关系。
- 提示 2: 考虑基于值域进行计数。

ARC148E $\geq K$

考虑将该数列分为两部分： $< \frac{K}{2}$ 和 $\geq \frac{K}{2}$ ，我们分别称其为 A 类与 B 类。

考虑拼接关系，A 类数与 A 类数一定不能拼接，B 类数与 B 类数一定可以拼接。那么我们可以将问题看作，先将 B 类数随机排列，然后在其中插入 A 类数，每一个空只能插入一个 A 类数，求方案数。

ARC148E $\geq K$

考虑 A 类数与 B 类数之间的关系，可以发现，将两类数分别排序后，可以与 A 类数拼接的 B 类数一定是一段后缀，我们可以借助该关系推导出基于值域的做法。

考虑从大到小将 B 类数插入序列，在插入值 b 之前将所有没有被插入的 $a + b < K$ 插入进序列，这样就只需要保证后续插入的 B 类数不与先前插入的 A 类数相邻即可。

考虑每一步时的方案数，假设当前序列中有 x 个 A 类数与 y 个 B 类数，新插入任意一个数都要求不与先前插入的 A 类数相邻，因此可插入的位置个数为 $y + 1 - x$ 。

将每一步的方案数直接相乘就是总方案数了。注意对于相同的数在算完答案后需要取消顺序。

QOJ9489 0100 Insertion

给定一个长度为 n 的包含 0、1、? 的字符串，将每个 ? 替换为 0 或 1，求得到的合法字符串个数。一个字符串合法当且仅当其可以通过不断删除子串 0100 删空。

数据范围： $4 \leq n \leq 500$ 。

QOJ9489 0100 Insertion

- 提示 1: 考虑对 01 进行加权。

QOJ9489 0100 Insertion

- 提示 1: 考虑对 01 进行加权。
- 提示 2: 考虑通过各种方法 (比如折线图) 刻画删除过程, 确定删除顺序。

QOJ9489 0100 Insertion

- 提示 1: 考虑对 01 进行加权。
- 提示 2: 考虑通过各种方法 (比如折线图) 刻画删除过程, 确定删除顺序。
- 提示 3: 写一个 dp。

首先很容易观察出来任意两个 1 不能相邻，这就代表 0100 前三位其实已经就位了，只需要最后一个 0 就可以删除。由此可以发现我们需要在合适的位置保留下 0。

QOJ9489 0100 Insertion

容易发现一种策略为找到所有 -3 在折线图中高度最高的进行优先的删除目标，若该位置不存在最后一个 $+1$ ，则需要通过不断删除后面的 -3 来将该位置缺失的 $+1$ 补充。由于该策略不会将一个本应该存在的 $+1$ 凭空删除，因此该删除方法是最优的。这时候再来考虑充要条件，可以发现我们只需要保证对于每一个 -3 能够找到其后面高度对应高度的 $+1$ 即可，因此可以得到充要结论：

对于每一个 $-3 : x \rightarrow x - 3$ ，满足在该 -3 后面的折线最高点 $\geq x - 1$ 。

QOJ9489 0100 Insertion

写一个 dp, 从后往前转移, $f_{i,j,k}$ 表示考虑了第 i 位往后的字符串, 当前折线高度位置为 j , 折线高度最高值为 k 的方案数。
 每一次转移可以往前扩展字符串 0 或者 01, 用以保证序列中不存在相邻的 1。

P11346 / QOJ6333 会议室 2

给定 n 个端点互不相同的区间，定义区间集合的权值为所有区间的并集的段数。你可以执行 $n - 1$ 次删除操作，每次可以将一个区间删除，求出有多少不同的删除方案，使得每一次删除后剩余的区间集合的权值和最小。

数据范围： $2 \leq n \leq 2000$ 。

P11346 / QOJ6333 会议室 2

- 提示 1: 删除 $\rightarrow$ 加入。

P11346 / QOJ6333 会议室 2

- 提示 1: 删除 $\rightarrow$ 加入。
- 提示 2: 维护并集。

P11346 / QOJ6333 会议室 2

- 提示 1: 删除 $\rightarrow$ 加入。
- 提示 2: 维护并集。
- 提示 3: 考虑 top 序计数。

P11346 / QOJ6333 会议室 2

这就启发我们去维护当前并集，定义 $f_{l,r}$ 表示当前已经加入到并集为 $[l, r]$ 的方案数。但是有一个严重的问题是有一些区间可能还尚未加入，并且其完全被 $[l, r]$ 包含，这就导致的不合法的转移。考虑解决这个问题，对于 f 的转移，我们只考虑使得并集产生变化的区间加入，对于其余区间的加入可以使用组合数学进行计算。

数据范围很小，我们可以对于每一区间 $[l, r]$ 求出有多少区间被 $[l, r]$ 包含，不妨设为 $v_{l,r}$ 。

P11346 / QOJ6333 会议室 2

那么情况就是，在进行转移 $f_{l',r'} \leftarrow f_{l,r}$ 的时候，除去扩展并集的这一个区间，其余的可以任意时刻加入的区间总共新增了

$v_{l',r'} - v_{l,r} - 1$ 。这个形式容易让我们联想到树的 top 序计数。

我们不妨考虑每一条转移链 $f_{l_1,r_1} \rightarrow f_{l_2,r_2} \rightarrow \cdots \rightarrow f_{l_k,r_k}$ ，我们将每一个 dp 状态视作一个节点，而这些节点间连成链状结构，在这些节点下还分别连接有额外的

$v_{l_1,r_1} - 1, v_{l_2,r_2} - v_{l_1,r_1} - 1, \cdots, v_{l_k,r_k} - v_{l_{k-1},r_{k-1}} - 1$ 个节点，最后我们计算方案相当于在对这棵树以 f_{l_1,r_1} 为根的 top 序计数。

P11346 / QOJ6333 会议室 2

top 序计数的结论是 $n! \times (\prod siz_u)^{-1}$ ，在这里则是：

$$(n-1)! \times \left(\prod_i^{k-1} (n - v_{l_i, r_i}) \right)^{-1}$$

因此我们直接在 dp 的同时计算该式子即可，使用二维前缀和快速转移，时间复杂度为 $O(n^2)$ 。

AGC020F Arcs on a Circle

有长度为 c 的圆圈与 n 条线段，线段的长度分别为 l_i ，考虑对于每条线段，分别将其随机放在圆圈的任意位置上，求所有线段覆盖了整个圆圈的概率。输出误差为 10^{-11} 。

数据范围： $2 \leq c \leq 50$, $2 \leq n \leq 6$, $1 \leq l_i < c$, 输入值均为整数。

AGC020F Arcs on a Circle

- 提示 1: 数据范围很小, 将连续的问题离散化。

AGC020F Arcs on a Circle

- 提示 1: 数据范围很小, 将连续的问题离散化。
- 提示 2: 数据范围真的很很小, 考虑状压 dp。

AGC020F Arcs on a Circle

我们首先规定圆圈上的一个零点（该点可以依附于任何地方，计算结果不变），定义圆上的点的距离为从零点开始沿着顺时针走到该点所需要的距离，定义一条线段在圆上的位置为其起点的距离。

随机放在任意位置看起来非常魔怔，但是输入的数都是整数，所以考虑将题离散化。将所有线段的距离拆为小数部分与整数部分，两条线段的相交关系可以通过计算整数部分的差与小数部分的相对关系判断。由于我们只需要小数部分的相对关系，我们可以直接暴力枚举所有 $(n - 1)!$ 种相对关系。

AGC020F Arcs on a Circle

接下来就是看起来比较正常的问题了，我们不妨假设最长的线段起点为零点，然后断环成链，转换为该问题：

除去最长的线段放置于 $[0, l)$ ，其余线段的放置位置为 $[x + rk_i \times \epsilon, x + l_i + rk_i \times \epsilon)$ ，其中 x 为 $[0, c)$ 内的整数， rk_i 为第 i 条线段的排名。求所有线段覆盖了 $[0, c)$ 的概率。

AGC020F Arcs on a Circle

考虑状压 dp 求解该问题，设 $dp_{S, i_1, i_2, j_1, j_2}$ 表示目前线段的放置情况为 S ，当前已经考虑了 $i_1 + i_2 \times \epsilon$ 前的决策，目前线段覆盖的最远点为 $j_1 + j_2 \times \epsilon$ ，求方案数。

转移非常简单，只需要跳到下一个关键点，考虑该线段是否需要放下，同时维护最远覆盖点即可。该部分复杂度为 $O(n^2 c^2 2^n)$ 。直接在外层枚举相对关系的复杂度为 $O((n-1)! n^2 c^2 2^n)$ ，将相对关系的枚举改为状压后复杂度为 $O(4^n n^3 c^2)$ 。出题人非常良心两种做法都可以通过。

另外还有使用生成函数的做法可以做到不依赖值域的复杂度。

QOJ6333 Festivals in JOI Kingdom 2

给定值域为 $[1, 2n]$ 的 n 个区间 $[l_i, r_i]$ ，使得端点互不相同，我们需要选出尽可能多的互不相交的区间。

有一个伪算为，每次选择左端点最小的，与已选区间互不相交的区间。求出这 $(2n-1)!!$ 种可能的区间组中，该伪算不是最优解的区间组的数量。

数据范围： $1 \leq n \leq 10000$ 。

QOJ6333 Festivals in JOI Kingdom 2

- 提示 1: 找到一个 std, 考虑将伪算区间与 std 区间之间的关系。

QOJ6333 Festivals in JOI Kingdom 2

- 提示 1: 找到一个 std, 考虑将伪算区间与 std 区间之间的关系。
- 提示 2: 注意你的转移方法, 不同的转移方向不一定有相同的结果。

QOJ6333 Festivals in JOI Kingdom 2

正难则反，计算该伪算是最优解的区间组数量。

我们设 std 为，每次选择右端点最小的，与已选区间互不相交的区间。

我们首先考虑当伪算是最优解时区间的性质，我们可以发现，如果将伪算的区间与 std 的区间一一对应，有如下性质：

- std 区间的右端点一定小于等于伪算区间的右端点 (std 的最优性)；
- std 区间的右端点一定大于伪算区间的左端点，否则 std 在取完该区间后直接取伪算的后半部分更优。

QOJ6333 Festivals in JOI Kingdom 2

因此, std 区间 $[l, r]$ 与伪算区间 $[\ell, r']$ 只有三种可能的相对关系:

- ① $\ell < l < r < r'$;
- ② $l < \ell < r < r'$;
- ③ $l = \ell < r = r'$ 。

QOJ6333 Festivals in JOI Kingdom 2

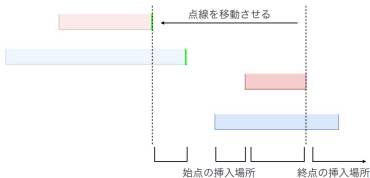
接下来考虑使用 dp 计算方案数，我们发现区间之间的限制非常复杂：伪算区间的限制需要满足其余区间的左端点不落在相邻的两个伪算之间，std 区间的限制则需要满足在左端点落在前一个区间的右端点后时，右端点在后一个区间的右端点后。我们可以发现这些限制对左端点的约束比右端点的约束强，因此我们可以考虑从后往前进行 dp，这样在转移过程中可以更方便的进行右端点的添加。反之则会遇到左端点约束过于复杂无法快速计算贡献的劣势。

QOJ6333 Festivals in JOI Kingdom 2

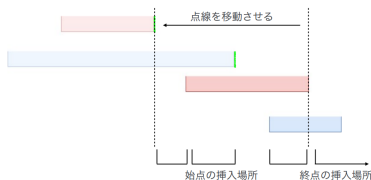
设 f_i 表示目前确定了 $i-2$ 个区间以及 2 个区间的右端点，这两个区间分别为 std 区间和伪算区间（对应前两种相对关系）的方案数。设 g_i 表示目前确定了 $i-1$ 个区间以及 1 个区间的右端点，这一个区间同时为 std 区间和伪算区间（对应第三种相对关系）的方案数。

接下来我们就需要对转移进行分讨，以 $f \leftarrow f$ 为例：

QOJ6333 Festivals in JOI Kingdom 2



(a) Case 1



(b) Case 2

图: 分讨

QOJ6333 Festivals in JOI Kingdom 2

这两种情况的结构基本一致，需要插入的两个左端点与两个右端点均为固定形式，其余区间的左端点的插入位置也均为三个。假设加入的其余区间数量为 j ，则此时的转移系数为：

$$\binom{j+2}{2} \binom{2i-3+j}{j} j! = \frac{(j+2)^2(2i-3+j)^j}{2}.$$

第二种情况的分析与第一种情况完全一致，转移系数也完全一致，所以结合两种情况，此部分的转移为：

$$f_{i+j+2} \leftarrow f_i \times (j+2)^2(2i-3+j)^j.$$

QOJ6333 Festivals in JOI Kingdom 2

其余转移的分析更为简便，此处略去分析：

$$f_{i+j+2} \leftarrow g_i \times (j+1)(2i-3+j)^j.$$

$$g_{i+j+1} \leftarrow f_i \times (j+1)(2i-2+j)^j.$$

$$g_{i+j+1} \leftarrow g_i \times (2i-2+j)^j.$$

复杂度为 $O(n^2)$ ，原题数据范围为 $n \leq 20000$ ，需要卡常或者使用卷积优化。

CF1844H Multiple of Three Cycles

给定一个长度为 n 序列 a ，初始为全空， n 次修改操作为将 a 的一个位置修改为一个数，保证所有修改后 a 为排列。

每次修改后，你要求出，所有将 a 补充为排列 a' 的方案中，排列的环长均为 3 的倍数的方案数。

数据范围： $3 \leq n \leq 3 \times 10^5$ 。

CF1844H Multiple of Three Cycles

- 提示 1: 考虑单组询问。

CF1844H Multiple of Three Cycles

- 提示 1: 考虑单组询问。
- 提示 2: 考虑使用组合意义求出一些结果之间的关联。

CF1844H Multiple of Three Cycles

- 提示 1：考虑单组询问。
- 提示 2：考虑使用组合意义求出一些结果之间的关联。
- 提示 3：在二维平面画一画使用到的信息，尝试构造一种递推转移方法。

CF1844H Multiple of Three Cycles

在去掉已经成环的数之后，我们求出答案实际上只需要知道在 $\text{mod } 3$ 的意义下链长分别为 0、1、2 的链个数，不妨假设其为 a, b, c 。

我们先来尝试求出单组询问，设 $f(a, b, c)$ 为该问题的答案，那么我们尝试使用组合意义找到一些恒等式用来推导出答案。

我们可以立即发现， $f(a, b, c) = (a + b + c)f(a - 1, b, c)$ ，原因是我们找到包含最小编号的 0 长链，我们发现这条链可以接在任意一条其余的链后面，或者自己成环，剩余的问题则是一个 $f(a - 1, b, c)$ 子问题。

CF1844H Multiple of Three Cycles

因此我们可以定义 $g(b, c) = f(0, b, c)$, 那么有
 $f(a, b, c) = (a + b + c)^a g(b, c)$, 那么我们再来看看关于 $g(b, c)$ 的一些恒等式。

$$\begin{aligned} g(b, c) &= (b-1) \times g(b-2, c+1) + c \times f(1, b-1, c-1) \\ &= (b-1) \times g(b-2, c+1) + c(b+c-1) \times g(b-1, c-1). \\ g(b, c) &= (c-1) \times g(b+1, c-2) + b \times f(1, b-1, c-1) \\ &= (c-1) \times g(b+1, c-2) + b(b+c-1) \times g(b-1, c-1). \end{aligned}$$

CF1844H Multiple of Three Cycles

例如，假如说我们知道了 $g(a, b)$ 与 $g(a + 1, b + 1)$ ，那么我们可以通过如下过程构造出 $g(a + 2, b + 2)$ ：

- ① 通过恒等式 1，使用 $g(a, b)$ 与 $g(a + 1, b + 1)$ 求得 $g(a - 1, b + 2)$ ；
- ② 通过恒等式 2，使用 $g(a - 1, b + 2)$ 与 $g(a + 1, b + 1)$ 求得 $g(a, b + 3)$ ；
- ③ 通过恒等式 1，使用 $g(a, b + 3)$ 与 $g(a + 1, b + 1)$ 求得 $g(a + 2, b + 2)$ 。

CF1844H Multiple of Three Cycles

此方法的缺点是计算过程中涉及到了逆元操作，因此所有的恒等式都只能在逆元存在的情况下使用，需要对边界情况进行特判。

求单组询问的复杂度为 $O(b + c)$ 。

题目所问的是多组询问，但是由于修改操作至多只会对 a, b, c 每个数造成至多 $O(1)$ 的影响，因此回答每组询问的复杂度是 $O(1)$ 。总复杂度为 $O(n + \log \text{mod})$ 。

CF2135E2 Beyond the Palindrome

对于 01 串 r , 我们定义 $f(r)$ 为重复删除其所有 10 子串直到无法删除后的到的结果。

多组测试, 给定正整数 n , 求出长度为 n 的 01 串 s 的数量, 满足 $f(s) = f(\text{rev}(s))$, 其中 $\text{rev}(s)$ 表示 s 的翻转。

数据范围:

- Easy Version: $\sum n \leq 10^6$ 。
- Hard Version: $n \leq 2 \times 10^7, \sum n \leq 10^8$ 。

CF2135E2 Beyond the Palindrome

- 提示 1: 将字符串转为折线图, 解读题目中的限制。

CF2135E2 Beyond the Palindrome

- 提示 1: 将字符串转为折线图, 解读题目中的限制。
- 提示 2: 当我们同时知道 01 个数之差以及折线图极差, 应该怎么做? 使用容斥降低思考难度。

CF2135E2 Beyond the Palindrome

- 提示 1: 将字符串转为折线图, 解读题目中的限制。
- 提示 2: 当我们同时知道 01 个数之差以及折线图极差, 应该怎么做? 使用容斥降低思考难度。
- 提示 3: 当我们只知道折线图极差, 应该怎么做?

CF2135E2 Beyond the Palindrome

我们将长度为 n 的 01 串转换为折线图，0 代表 $+1$ ，1 代表 -1 。那么容易发现 $f(s)$ 的结果为若干个 0 接若干个 1，其中 0 的个数为折线图的最大值减去起点，1 的个数为折线图的最大值减去终点。

那么 $f(s) = f(\text{rev}(s))$ 就指的是：设起点为 s ，终点为 t ，折线图最大值为 mx ，折线图最小值为 mn ，则 $s + t = mx + mn$ 。

CF2135E2 Beyond the Palindrome

我们先解决一个类似的问题，从 s 走 n 步到 t ，期间路径不能接触到 u 与 d 两个边界 ($u > d$)，求合法的方案数。

我们可以使用叫做反射容斥的 trick 解决该问题。考虑该路径可能会接触到 u 与 d 两个边界，我们给每一条可能的路径赋予一个标识，当接触到上边界时标识末尾插入 A，接触到下边界时标识末尾插入 B，最后进行相邻的去重。

由该标识的定义，我们需要求出的是标识等于 $\emptyset$ 的路径个数，此时我们通过容斥来进行计算，定义 $\Delta = u - d$ 。

CF2135E2 Beyond the Palindrome

不断的容斥下去，最终结果如下：

- 加上所有从 $2k\Delta + s$ 走到 t 的方案数，
- 减去所有从 $2k\Delta + 2u - s$ 走到 t 的方案数。

路径方案数可以使用组合数 $O(1)$ 计算，最终的答案如下：

$$\sum_k \binom{n}{(t-s+n)/2 + k\Delta} - \binom{n}{(t-2u+s+n)/2 + k\Delta}.$$

时间复杂度为 $O(n/\Delta)$ 。

CF2135E2 Beyond the Palindrome

解决该问题后，当我们同时知道 01 个数之差以及折线图极差时，我们只需要再次使用容斥，求

$u \in \{mx + 1, mx\}, d \in \{mn - 1, mx\}$ 四组问题就可以算出答案。

暴力枚举个数差与极差的时间复杂度为 $O(n^2)$ 。

为了方便描述，我们把原问题容斥，变为枚举 01 个数之差与 Δ ，对满足 $s + t = u + d + c$ 的路径数计数 ($c \in \{0, 1\}$)。

CF2135E2 Beyond the Palindrome

在枚举 Δ 之后，改变 01 个数之差相当于改变了 $s - t$ ，经过不难的推导可以发现在固定上述公式中的 k 之后，两组合数均可以使用区间和求得，复杂度为 $O(n \ln n)$ ，可以通过 Easy Version。我们首先可以发现，当 $s + t = u + d + c$ 时，公式中的信息可以再次化简，设 $c_0 = (t - s + n)/2$ 为原字符串中 0 的个数，则公式可化为：

$$\sum_k \binom{n}{c_0 + k\Delta} - \binom{n}{\frac{n-\Delta+c}{2} + k\Delta}.$$

CF2135E2 Beyond the Palindrome

注意我们在枚举 Δ 后需要满足 $|t - s| \leq \Delta - c - 2$, 得到 $\frac{n-(\Delta-c)}{2} < c_0 < \frac{n+(\Delta-c)}{2}$ 。我们发现, 第一个组合数在限定的 c_0 下会有一些缝隙, 而第二个组合数几乎完美的塞下了缝隙。最终的式子如下:

$$2^n - \Delta \sum_k \binom{n}{\frac{n-\Delta+c}{2} + k\Delta}.$$

那么现在的问题变为如何计算后半节式子, 我们现在可以尝试去找到每一个组合数 $\binom{n}{i}$ 的系数 c_i 。

CF2135E2 Beyond the Palindrome

设 g_i 为:

$$g_i = \sum_{\Delta} \Delta \left[\frac{i}{\Delta} = 2k + 1 \right].$$

那么可以看出来, g_i 相当于是枚举了 i 所有的因数 Δ , 并且将所有 i/Δ 为奇数的 Δ 相加。这个函数在排除掉质因子 2 之后是积性函数, 可以使用线性筛求解。

总时间复杂度为 $O(n)$ 。

AGC053E More Peaks More Fun

给定 n 对数 (a_i, b_i) , 对于所有 $n!$ 种排列这些对的方案, 求出有多少方案合法:

一个排列方案 $(a'_1, b'_1), \dots, (a'_n, b'_n)$ 合法当且仅当, 构造一个序列 $f = [a'_1, b'_1, a'_2, b'_2, \dots, a'_n, b'_n]$:

存在一种任意交换 f_{2k-1}, f_{2k} 的方案使得总共有 $n - 1$ 个 i 满足 $2 \leq i \leq 2n - 1, f_{i-1} < f_i > f_{i+1}$ (山峰)。

$1 \leq n \leq 10^6, 1 \leq a_i, b_i \leq 2n$ 且 a_i, b_i 互不相同。

AGC053E More Peaks More Fun

- 提示 1: $n - 1$ 这个数是长度为 $2n$ 的序列的最大可能山峰数量, 考虑可能的山峰分布。

AGC053E More Peaks More Fun

- 提示 1: $n - 1$ 这个数是长度为 $2n$ 的序列的最大可能山峰数量, 考虑可能的山峰分布。
- 提示 2: 考虑简单的情况: 只算第 $2, 4, \dots, 2n - 2$ 位为山峰的方案数如何计数?

AGC053E More Peaks More Fun

我们称满足 $2 \leq i \leq 2n - 1, f_{i-1} < f_i > f_{i+1}$ 的 i 为山峰，不难发现山峰不可能相邻，因此山峰的最大个数就是 $n - 1$ 。
 当山峰个数为 $n - 1$ 时，不妨将序列 f 两两分组，可以发现其山峰的分布类似于这样：

$$[(0, 1), (0, 1), \dots, (0, 1), (0, 0), (1, 0), (1, 0), \dots, (1, 0)].$$

我们设中间 $(0, 0)$ 所在的位置为第 p 对，为了保证不算重，我们对于每种排列方案只在最大的 p 计算。

AGC053E More Peaks More Fun

不妨先保证 $a_i < b_i$, 我们先尝试计算 $p = n$ 的答案, 其充要条件为 $\forall 1 \leq i < n, b_i > a_{i+1}$ 。

考虑如何计数, 我们可以使用值域维度计算贡献, 具体来说将所有对 (a, b) 按照 b 进行从大到小排序, 然后依次插入答案序列。

当我们插入 (a, b) , 因为这个位置与前一个位置已经自然满足了 $b' > b > a$ 的条件, 实际上我们只需要去处理与后一个位置

(a', b') 的约束 $b > a'$ 。

也就是我们需要对每一对 (a, b) 计算出满足 $a' < b < b'$ 的 (a', b') 对数, 不妨假设其为 c , 那么加上将 (a, b) 插入末尾的方案, 每一个数对插入的方案数就是 $c + 1$ 。这个问题的答案也就是 $\prod (c + 1)$ 了。

c 可以使用二维数点或差分轻松算出。

AGC053E More Peaks More Fun

接下来考虑 $p < n$ 时答案的计算，首先我们需要保证该排列在 $p = n$ 时不合法，也就是存在 i 使得 $a_i < b_i < a_{i+1} < b_{i+1}$ 。我们找到所有不合法的位置 i ，然后取出最小的 i ，我们容易说明要么该排列方案不合法，要么该排列方案 p 等于这个最小的 i 。此时答案的约束为：

- $\forall 1 \leq i < p, b_i > a_{i+1};$
- $\forall p + 1 \leq i < n, a_i < b_{i+1}.$

AGC053E

我们不妨直接枚举第 $p, p+1$ 对的具体数值 $(a_x, b_x), (a_y, b_y)$, 满足 $b_x < a_y$, 那么沿用上述思路, 我们可以总结出每一对 (a, b) 的贡献:

- 当 $b_y < b$ 时, 贡献为 $c+2$;
- 当 $b_x < b < b_y$ 时, 贡献为 $c+1$;
- 当 $b < b_x$ 时, 贡献为 c 。

这一堆贡献也可以使用二维数点或差分轻松算出。
时间复杂度为 $O(n)$ (使用差分)。

AGC055F Creative Splitting

对于长度为 k 值域为 k 的序列 a , 定义其是好的当且仅当

$$\forall i, a_i \leq i.$$

对于长度为 nk 值域为 k 的序列 a , 定义其是完美的当且仅当其可以拆分为恰好 n 个好序列。

对于每一个 $1 \leq i \leq nk, 1 \leq j \leq k$, 求出有多少完美序列 a 使得 $a_i = j$ 。

数据范围: $1 \leq n, k \leq 20$ 。

AGC055F Creative Splitting

- 提示 1: 正难则反。

AGC055F Creative Splitting

- 提示 1: 正难则反。
- 提示 2: 尝试计算总方案数, 打表, 列出网格后, 考虑推导一些等价的形式, 使方案数变得更好算。

AGC055F Creative Splitting

我们需要知道如何判定一个序列 a 是好序列。
经过若干尝试可以发现，正着做半点前途没有，考虑反过来，相当于维护了长度为 n 的序列 c ，初始值为 k ，每遇到一个数 a 需要找到一个 $c_i \geq a$ ，并将其减一，如果找不到则不合法。不难证明这里选择最小合法的 c_i 进行操作一定更优。

AGC055F Creative Splitting

我们接下来考虑在不限制 $a_i = j$ 的情况下答案是多少。

通过打表可以得到 $(nk)!/(n!)^k$ 。

此时我们不妨假设 c 是从大到小进行排序的，每一次操作优先对下标更大的数进行操作，那么我们在 $n \times k$ 的网格中画出被占领的格子，其形成了位于右下角的小坡。在进行下一次操作的时候，可以视作有一个标记从第 a 列下降，直到落到了一个已经占领的格子上，然后向右移动直到挨着墙，最后将该标记所在格子占领，代表对应行的 c 减一。

AGC055F Creative Splitting

然后我们发现一个惊人的事情：我们其实没有必要进行向右移动这个操作，因为从列的视角来看，每一列所占领的格子数的集合在向右移动的过程中一定不会变化。因此不进行向右移动所算出来的方案数跟原问题方案数是一样的。那么每一次选择一个列进行下落，就很容易得到 $(nk)!/(n!)^k$ 这个结果了。

AGC055F Creative Splitting

考虑扩展到原问题，先设 $d_i = \sum [c_j < i]$ 表示每列被占领的格子数。假如说我们现在需要计算 $a_x = y$ 的方案数，那么我们的实际步骤如下：

- ① 枚举 d 数组，使得 $\sum d = nk - x$ ，表示在 a_x 之后的数的占领情况；
- ② 将 d 数组从大到小排序为 d' ，目的是检验 $a_x = y$ 能否成立，即是否有 $d'_y < n$ ；
- ③ 通过 d 计算出在 a_x 之后与在 a_x 之前的方案数并相乘。

AGC055F Creative Splitting

当我们枚举排序后的数组 d' 时条件更好判断，此时的方案数为：

$$\sum_{\substack{d' = nk - x \\ d'_1 \leq d'_2 \leq \dots \leq d'_k \\ d'_y < n}} \frac{k!}{\prod_i (\sum_j [d'_j = i])!} \frac{(nk - x)!}{\prod_i d'_i!} \frac{(x - 1)!}{\prod_i (n - d'_i - [i = y])!}$$

第一个分式表示将 d' 转换为 d 的方案数，第二个分式表示 a_x 之后的方案数，第三个分式表示 a_x 之前的方案数。

AGC055F Creative Splitting

但是直接枚举过不去，我们需要使用 dp，定义 $f_{i,j,s}$ 表示，当前已经枚举了 $d' \geq i$ 的情况，当前已经填写了 j 个数，总和为 s 的情况下， $(\prod_i (\sum_j [d'_j = i])!) (\prod_i d'_i! (n - d'_i - [i = y])!)$ 的贡献。在进行 dp 转移的时候需要注意所有 $f_{n,j,nj}$ ($j > k - y$) 的状态都是不合法的，因为违反了 $d'_y < n$ 的限制。

直接暴力枚举 x, y 再暴力转移复杂度是 $O(n^3 k^5)$ ，但实际上 dp 的过程只需要涉及到 y ，因此先枚举 y 之后预处理 dp 再枚举 x ，复杂度就变为了 $O(n^2 k^4)$ 。